

DSA & Advanced DSA Roadmap

Phase 15 — Greedy Algorithms

Learning Objectives

By the end of this phase, learners will be able to:

- Explain the greedy choice property: making the locally optimal choice at each step, without reconsidering past choices, and why this only produces a globally optimal result for certain problems.
- Distinguish problems where greedy is provably optimal from problems where it merely seems plausible but actually requires dynamic programming (Phase 16) for a correct answer.
- Apply interval-scheduling greedy strategies, choosing the correct sort key (start time vs. end time vs. value) based on the specific objective.
- Implement the fractional knapsack and Huffman coding algorithms from scratch, and explain why each is greedy-optimal while the related 0/1 knapsack problem is not.
- Solve Easy to Medium level greedy problems independently, and approach Hard scheduling problems by identifying the right greedy invariant to maintain.

Implementation & Internals

Core Concepts

- Greedy Choice Property — a problem has this property if a sequence of locally optimal choices, each made without reconsidering earlier ones, leads to a globally optimal solution.
- Local vs Global Optimum — the central risk of greedy algorithms: a choice that looks best right now can permanently rule out the actual best overall solution. Proving a greedy strategy correct (e.g. via an exchange argument) is what separates greedy algorithms from 'algorithms that happen to look greedy.'

Implementation Exercises

- Implement Fractional Knapsack from scratch: given item values and weights and a capacity, sort items by value-to-weight ratio descending, then greedily take whole items until the next item must be split fractionally.
- Implement Huffman Coding from scratch: given character frequencies, repeatedly combine the two least-frequent nodes into a new parent node (using a min-heap, directly reusing Phase 12's heap implementation) until one tree remains, then derive each character's binary code from its path to the root.
- Construct a small counterexample showing that a naive greedy strategy fails on the 0/1 knapsack problem (where items cannot be split), to make explicit why fractional knapsack's greedy approach does not transfer to that closely related problem.

Pattern Mastery

Pattern 1: Interval Scheduling

Concepts

- Sorting intervals (by start, end, or value depending on the objective) and greedily selecting which to keep, directly extending Phase 8's Interval Sorting pattern with an explicit greedy-choice justification.

- Recognizing the 'attend as many events as possible' shape: at each day, greedily attend the available event with the earliest deadline, since deadlines that are far away can always be served later.

Practice Problems

- Given a list of events, each with a start and end day, find the maximum number of events that can be attended by attending at most one event per day.

LeetCode

- [Maximum Number of Events That Can Be Attended — leetcode.com/problems/maximum-number-of-events-that-can-be-attended](https://leetcode.com/problems/maximum-number-of-events-that-can-be-attended/)

Note: this phase's interval-scheduling problem is intentionally different from Phase 8's Interval Sorting problems (Meeting Rooms II, Non-overlapping Intervals, Minimum Number of Arrows to Burst Balloons), which already covered the sort-by-end-time greedy shape. This problem instead requires a day-by-day greedy simulation with a min-heap of deadlines.

Pattern 2: Meeting Rooms

Concepts

- Simulating real-time room assignment under specific tie-breaking rules (lowest room number first, earliest original start time first when delayed), extending the room-counting idea from Meeting Rooms II (Phase 8) into a full assignment simulation.
- Using two heaps — one for idle rooms ordered by room number, one for busy rooms ordered by free time — to make each greedy assignment decision in $O(\log n)$.

Practice Problems

- Given n meeting rooms and a list of meetings, simulate room assignment under specific rules and return which room hosted the most meetings.

LeetCode

- [Meeting Rooms III — leetcode.com/problems/meeting-rooms-iii](https://leetcode.com/problems/meeting-rooms-iii/)

Note: Meeting Rooms (Phase 1, flagged as Premium-only) and Meeting Rooms II (Phase 8) are not repeated here. Meeting Rooms III is a substantially different, free problem: a full simulation with delay and tie-breaking rules, not just a room count.

Pattern 3: Fractional Knapsack

Concepts

- Sorting items by value-to-weight ratio, then greedily filling the knapsack with whole items until the last one must be split fractionally to exactly fill remaining capacity.
- Understanding why this greedy approach is provably optimal here but not for 0/1 knapsack: fractional splitting removes the combinatorial 'which subset' decision that makes 0/1 knapsack require dynamic programming (Phase 16).

Practice Problems

- Implement the fractional knapsack algorithm from scratch and verify it against the classic worked example: items with values [60, 100, 120] and weights [10, 20, 30], capacity 50, expected output 240.

No LeetCode link: Fractional Knapsack is a classic algorithms-textbook problem (covered on GeeksforGeeks and in CLRS) without a free, native LeetCode equivalent. It is assessed here as a from-scratch implementation exercise instead.

Pattern 4: Huffman Coding

Concepts

- Building an optimal prefix-free binary code by repeatedly merging the two lowest-frequency nodes in a min-heap into a new parent node, until a single tree remains.
- Connecting directly back to Phase 12's heap implementation: Huffman coding is, structurally, the exact same 'always combine the two smallest' greedy shape used there, now applied to building a tree rather than computing a numeric total.

Practice Problems

- Implement Huffman coding from scratch: given a set of characters and their frequencies, build the Huffman tree and derive each character's binary code, then verify that more frequent characters receive shorter codes.

Phase Assessment

Implementation Tasks

- Implement Fractional Knapsack from scratch and verify against the classic worked example.
- Implement Huffman Coding from scratch, reusing the Phase 12 heap implementation, and verify that frequency ordering matches code-length ordering.
- Construct and explain a counterexample showing why the fractional knapsack greedy strategy fails if applied directly to 0/1 knapsack.

Recommended LeetCode Target

Medium Problems

Interval Scheduling

- leetcode.com/problems/maximum-number-of-events-that-can-be-attended

Hard Problems

- leetcode.com/problems/meeting-rooms-iii

On scope and dedup

This phase has only 2 new LeetCode problems — the smallest count in the roadmap — for two compounding reasons. First, 7 of the most natural greedy/interval problems (Meeting Rooms I/II, Non-overlapping Intervals, Minimum Arrows to Burst Balloons, Merge/Insert Intervals, Assign Cookies, Boats to Save People) were already used in Phases 1 and 8. Second, 2 of this phase's 4 named patterns (Fractional Knapsack, Huffman Coding) are classic algorithms-textbook problems with no free LeetCode equivalent, and are assessed as from-scratch implementation exercises instead. None of the 2 new problems overlaps with any problem used in Phases 0–14.

Coding Challenges (Assessment Day)

- Maximum Number of Events That Can Be Attended
- Meeting Rooms III
- Fractional Knapsack (from scratch)
- Huffman Coding (from scratch)

Expected Outcome

Students should be able to:

- Explain and prove (informally, via exchange argument) why a given greedy strategy is correct for a specific problem.
- Recognize the four greedy shapes covered: interval scheduling, real-time resource assignment, ratio-based item selection, and frequency-based merging.
- Distinguish problems that are genuinely greedy-solvable from superficially similar problems (like 0/1 knapsack) that require dynamic programming instead.
- Solve unseen greedy problems by first asking 'what is the greedy choice here, and can I justify it?' rather than assuming any locally-sensible strategy is correct.